



第三章 正则表达式（正规式）2026

赵伟



本章知识图谱

- RL:(识别器:(FA RE RG) 判定性质:($\tilde{v}$ 模式匹配 生成) 等价性质:(**RE $\equiv$ FA:(DFA2RE RE2NFA)** RE $\equiv$ RG NFA $\equiv$ DFA **DFA最小化**))
- RE:(生成器:(形式文法 regex) 代数性质)
- RL簇:(封闭性:(基本运算 运算) 泵引理 空性 成员性))



RL簇上的基本运算

- 术语：语言类与语言簇
 - 语言类：按计算能力（自动机、文法）严格分类
 - 经典分类：Chomsky 0-3型
 - 语言簇：按特征、场景 的松散归类
- RL簇:(封闭性:(**基本运算** 运算) 泵引理 空性 成员性))
- **基本运算**:
 - 闭包'*'（克林闭包、上标'*'）
 - 连接'◦'（常常省略）
 - 并'U'（集合并）



RL簇上的基本运算

- 令 L, L_1, L_2 是 Σ 上的语言。
- 连接运算：借助于符号串的连接运算计算结果
 - $L_1L_2 = \{st \mid s \in L_1, t \in L_2\}$
 - 语言连接是基于符号串连接，这两种运算符都被省略。
 - 该运算满足结合律： $L(L_1L_2) = (LL_1)L_2$
 - n 个 L 依次连接（ n 次幂） $L^n = \{s_1s_2...s_n \mid s_1, s_2, ..., s_n \in L\}$
 - $L^0 = \{\varepsilon\}$
- 克林闭包运算
 - $L^* = L^0 \cup L^1 \cup L^2 \cup ... = \bigcup_{n \in \mathbb{N}} L^n$
- 并运算 $\cup$ 就是集合并运算。



例：语言的连接运算、并运算

- $L_1 = \{0, 01\}$
- $L_2 = \{\varepsilon, 1, 11, 111, \dots\}$
- $L_1 L_2 = \{0, 01, 011, 0111, \dots\} \cup \{01, 011, 0111, \dots\}$
- $L_1^2 = \{00, 001, 010, 0101\}$
- $L_2^n = L_2 \ (n \geq 1)$
- $L_1 \cup L_2 = \{0, 01, \varepsilon, 1, 11, 111, \dots\}$



语言的闭包运算例

$$L_1 = \{0, 01\}, L_1^* = ?$$

L_1^* : 001000001属于 L_1^*

00110001不属于 L_1^*

10010001不属于 L_1^*

$$L_1^0 = \{\varepsilon\}$$

$$L_1^1 = \{0, 01\}$$

$$L_1^2 = \{00, 001, 010, 0101\}$$

$$L_1^3 = \{000, 0001, 0010, 00101, \\ 0100, 01001, 01010, 010101\}$$

L_1^* 所有以0开头不含连续1的串（加上空串）



语言闭包运算例

- 由任意个1组成的串的全集 L_2

$$L_2 = \{\varepsilon, 1, 11, 111, \dots\}, \quad L_2^* = ?$$

$$L_2^0 = \{\varepsilon\}$$

$$L_2^1 = L_2$$

$$L_2^2 = L_2$$

$$L_2^n = L_2 \quad (n \geq 1)$$

$$L_2^* = L_2^0 \cup L_2^1 \cup L_2^2 \cup \dots$$

$$= \{\varepsilon\} \cup L_2^1 \cup L_2^2 \cup \dots$$

$$= L_2$$

$$\varepsilon \in L^* ?$$

$$\emptyset^* = ?$$

$$\{\varepsilon\}^* = ?$$

$$\{1\}^* = L_2 ?$$

$$\varepsilon \in L^*$$

$$\emptyset^* = \{\varepsilon\}$$

$$\{\varepsilon\}^* = \{\varepsilon\}$$

$$\{1\}^* = L_2$$



构造语言的方法

- **原子语言**： Σ 上最基本的语言 $\emptyset$ 、 $\{\varepsilon\}$ 、 $\{a\}, a \in \Sigma$ 。
- **语言上运算**：**连接、闭包、并**。
- **构造方法**：从原子语言开始，利用语言上的运算逐步组合而成，其中可使用括号。
- **性质**：
 - 原子语言是 Σ 上语言。
 - 如果 L, L_1 是 Σ 上语言，那么 LL_1 、 $L \cup L_1$ 、 L^* 都是。
 - 任何 Σ 上语言的运算结果仍然是 Σ 上语言。
- **语言构造过程就是语言运算表达式的计算过程**。例子：
 - $L_1 = \{0\}(\{0\} \cup \{1\})^*$
 - $L_2 = (\{0\}\{1\}^*) \cup (\{1\}\{0\}^*)$
- 语言运算表达式**模式化为正则表达式**。



正则表达式的构造方法

(1) 原子语言的模式

- 语言 $\{a\}$ 模式化为RE a , 有 $L(a)=\{a\}$
- 语言 $\{\varepsilon\}$ 模式化为RE ε , 有 $L(\varepsilon)=\{\varepsilon\}$
- 空语言模式化为 $\emptyset$, 即 $L(\emptyset)=\emptyset$

RE运算符	语言运算符
连接'.'	连接'.'
并'+'	并'∪'
闭包'*'	闭包'*'

(2) 运算符 $\cup$ 改名为 $+$, 其它不变。

- 例: 语言 L_1 是0或由0开头的0-1串组成
 - 语言运算表达式: $\{0\}(\{0\}\cup\{1\})^*$
 - 对应正则表达式: $0(0+1)^*$
 - $L(0(0+1)^*) = \{0\}(\{0\}\cup\{1\})^* = L_1$
- 例: $L_2 = (\{0\}\{1\}^*) \cup (\{1\}\{0\}^*)$
 - 对应正则表达式: 01^*+10^*



构造正则表达式的规则

- 字母表 Σ 上的正则表达式是使用如下规则形成的表达式。
- 基础：**
 - 符号 $\emptyset$ 和 ε 都是正则表达式；
 - Σ 中每个元素 a 都是正则表达式；
- 归纳：**
 - 如果 R 和 S 是正则表达式，那么 $R+S$ 、 RS 和 R^* 都是；
 - 如果 R 是正则表达式，那么 (R) 也是。
- 正则表达式运算符的优先级
 - 优先级： '*' 最高， '.' 次之， '+' 最低
 - 计算顺序： 先括号里，后括号外，同级从左往右

a) $0(0 + 1)^*$

b) $01^* + 10^*$

c) $(0 + 1)^*01(0 + 1)^*$



RE生成器 (用文法生成RE)

- $R \rightarrow \emptyset \mid \varepsilon \mid a \mid RR \mid R^* \mid R+R \mid (R)$
- 例：生成一个正规式 $r=(0|1)^*11(0|1)^*$
- $R \Rightarrow RR \Rightarrow R^* R \Rightarrow (R)^* R \Rightarrow (R+R)^* R \Rightarrow (0+R)^* R \Rightarrow (0+1)^* R$
- $\Rightarrow (0+1)^* RR \Rightarrow (0+1)^* 1R \Rightarrow (0+1)^* 1RR \Rightarrow (0+1)^* 11R$
- $\Rightarrow (0+1)^* 11R^* \Rightarrow (0+1)^* 11(R)^* \Rightarrow (0+1)^* 11(R+R)$
- $\Rightarrow (0+1)^* 11(0+R)^* \Rightarrow (0+1)^* 11(0+1)^*$
- 正规式的构成：
 - 原子： $\emptyset$ 、 ε 、 a （指字母表中任意一个符号）
 - 运算符：闭包 * （上标）、连接 $''$ （省略）、并 $+$ （常用 $|$ ），优先级依次降低。
 - 园括号：优先级最高。



RE的定义

- $R \rightarrow \emptyset \mid \varepsilon \mid a \mid RR \mid R^* \mid R+R \mid (R)$
- 又例：生成一个正规式 $r=(0+1)^*101$
- $R \Rightarrow RR \Rightarrow R^* R \Rightarrow (R)^* R \Rightarrow (R+R)^* R \Rightarrow (0+R)^* R \Rightarrow (0+1)^* R$
- $\Rightarrow (0+1)^* RR \Rightarrow (0+1)^* 1R \Rightarrow (0+1)^* 1RR \Rightarrow (0+1)^* 10R$
- $\Rightarrow (0+1)^* 101$
- 由原子、运算符、括号连接起来的式子。类似算术表达式那样
- 事实上该形式文法定义RE为字母表上（ a 指代字母表符号，如上例中 a 指代0或1）正则表达式全集。
- RE的每一个成员是一个RL识别器。
- 对于一个具体的RE r ，它所定义的语言用 $L(r)$ 表示。



模式匹配：对于RE r 求 $L(r)$

- RL:(识别器:(FA **RE** RG) 判定性质(δ **模式匹配** 生成) 等价性质 ($RE \equiv FA$ $RE \equiv RG$ $NFA \equiv DFA$))
- RE:(生成器: $\{R \rightarrow \emptyset \mid \varepsilon \mid a \mid RR \mid R^* \mid R+R \mid (R)\}$ 代数性质)
- 模式匹配规则：
 - $L(\emptyset) = \emptyset$; $L(\varepsilon) = \{\varepsilon\}$; $L(a) = \{a\}$, $a \in \Sigma$
 - $L(r_1 r_2) = L(r_1)L(r_2)$
 - $L(r_1 + r_2) = L(r_1) \cup L(r_2)$
 - $L(r^*) = L(r)^*$
 - $L((r)) = L(r)$



例：给定RE r 求 $L(r)$

- $L(\emptyset) = \emptyset; L(\varepsilon) = \{\varepsilon\}; L(0) = \{0\}; L(1) = \{1\}$
- $L(r_1 r_2) = L(r_1)L(r_2)$
- $L(r_1 + r_2) = L(r_1) \cup L(r_2)$
- $L(r^*) = L(r)^*$
- $r = (0+1)^*11(0+1)^*$
- $L(r) = \{0,1\}^*\{11\}\{0,1\}^* = \text{“含有子串11的0-1串全集”}$
- $L((0+1)^*101) = \text{“以101结尾的0-1串全集”}$



RE的代数性质 (一)

- 交换律 (并) $R+S = S+R$
- 结合律 (并) $(R+S)+T = R+(S+T)$
- 结合律 (连接) $(RS)T = R(ST)$
- 左分配律 $R(S+T) = RS+RT$
- 右分配律 $(S+T)R = SR+TR$
- 分配律 (闭包) $(R+S)^* = (R^*S^*)^* = (R^*+S^*)^*$

- 反例 (无分配律) : $R+ST \neq (R+S)(R+T)$
- $R \rightarrow \emptyset \mid \varepsilon \mid a \mid RR \mid R^* \mid R+R \mid (R)\}$



RE的代数性质 (二)

- $R \rightarrow \emptyset \mid \varepsilon \mid a \mid RR \mid R^* \mid R+R \mid (R)\}$
- 恒等律 $R + \emptyset = R;$
- 恒等律 $\varepsilon R = R\varepsilon = R$
- 零元律 $\emptyset R = R\emptyset = \emptyset$
- 幂等律 $R \mid R = R;$
- 幂等律 $(R^*)^* = R^*;$
- 幂等律 $\emptyset^* = \varepsilon;$
- 幂等律 $\varepsilon^* = \varepsilon$
- 闭包的空串等价: $R^* = \varepsilon + R R^*$
- 闭包的左 / 右吸收律: $R^* R = R R^* = R^+$ (R^+ 是正闭包)



示例证明过程： P63 习题6 (2)

- R 是任意正则表达式，证明 $(R^*)^* = R^*$
- 只须证明 $L((R^*)^*) = L(R^*)$

对任意 $x \in L(R^*)$ ， $x = R^k$ ，其中 k 为非负整数；
那么 x 可写成 $(R^k)^1$ ，即与 $RE (R^*)^*$ 匹配；
因此， $x \in L((R^*)^*)$ 。

对任意 $x \in L((R^*)^*)$ ， $x = (R^k)^j$ ，其中 k 和 j 为非负整数；
那么 x 可写成 $R^{k \times j}$ ，其中 $k \times j$ 为非负整数；
所以 x 与 $RE R^*$ 匹配，从而， $x \in L(R^*)$ 。



例：分析正则表达式

- 以下例子都是关于RE的语言

$0+1$

长度为1的0-1串 $=\{0, 1\}$

$(0+1)^*$

任意0-1串 $\{\varepsilon, 0, 1, 00, 01, 10, 11, \dots\}$

$L((0+1)^*)=\{0,1\}^*$

$(0+1)^*010$

以010结尾的0-1串

$(0+1)^*11(0+1)^*$

包含子串11的0-1串



例：分析正则表达式

例： $((0+1)(0+1))^* + ((0+1)(0+1)(0+1))^*$

长度是偶数或是3的倍数的0-1串

$((0+1)(0+1))^*$

偶数长度

$(0+1)(0+1)$

长度为2

$((0+1)(0+1)(0+1))^*$

长度3的倍数

$(0+1)(0+1)(0+1)$

长度为3



例：分析正则表达式

$$((0+1)(0+1)+(0+1)(0+1)(0+1))^*$$

串可以分成段，
其中每段长度为**2**或**3**

$$(0+1)(0+1)+(0+1)(0+1)(0+1)$$

串长度为**2**或**3**

$$(0+1)(0+1)$$

长度**2**

$$(0+1)(0+1)(0+1)$$

长度**3**



例：分析正则表达式

$$(1+01+001)^*(\varepsilon+0+00)$$

以最多两个0结束

连续的1之间被最多两个0分开

不会有连续三个0的串

$$L((1+01+001)^*(\varepsilon+0+00)) = \{x: x \text{ 不含子串 } 000\}$$

ε

00

01100101110

0010010



例：写出正则表达式

- 该语言元素包含子串00
 - $(0+1)^*00(0+1)^*$
- 该语言元素不含子串00
 - $1^*(011^*)^*(\varepsilon + 0)$

111|011|01|011|01|01|0

开头有些 1 每个 0 后跟 有可能以一个 0 为结束
1 到多个 1

- 可有全 1 前缀，也可没有
- 中间部分，每个 0 后跟 1 到多个 1
- 可有 0 为后缀，也可没有



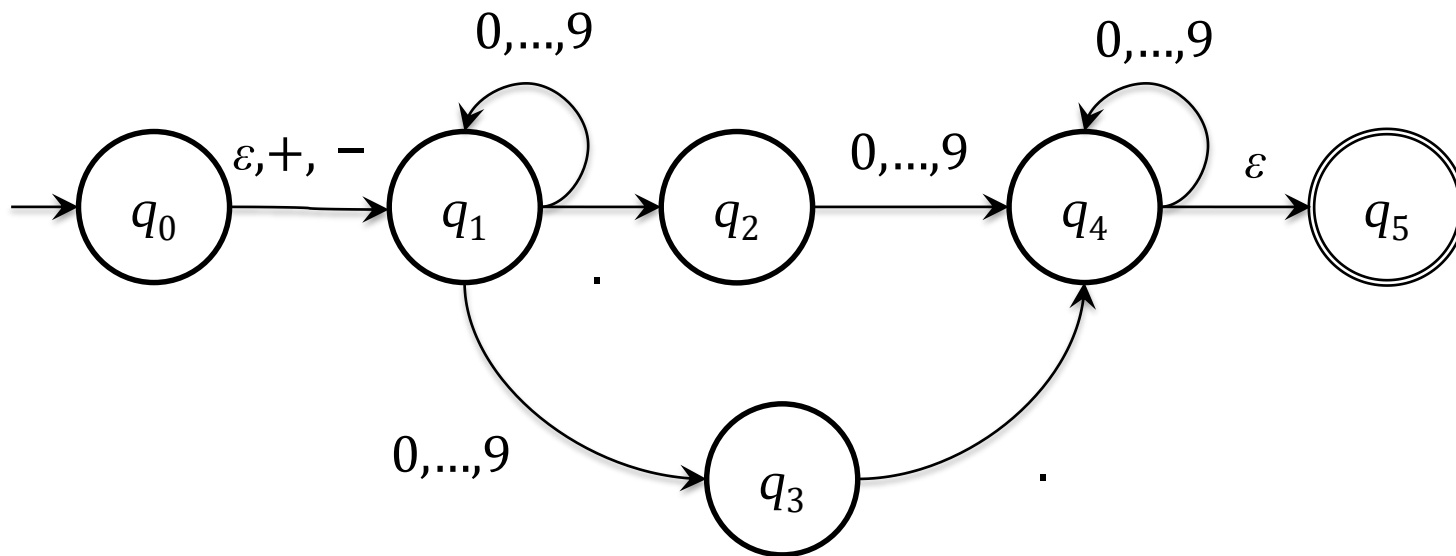
例：写出正则表达式

- 该语言元素都有偶数个0
 - $(1^*01^*01^*)^*$
- 该语言元素有偶数个0，或者有奇数个1
 - $(1^*01^*01^*)^* + (0^*10^*)(0^*10^*10^*)^*$
- 该语言元素有偶数个0，并且有奇数个1
 - ?



例：写出正则表达式

- 有符号十进制定点数。

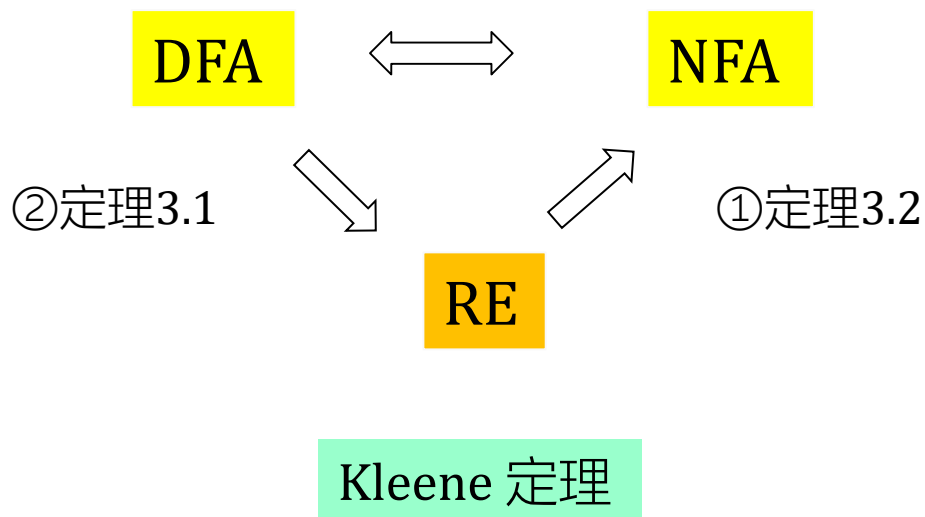


- $(\epsilon + \backslash + + -)((0 + \dots + 9)^* \cdot (0 + \dots + 9)(0 + \dots + 9)^* + (0 + \dots + 9)(0 + \dots + 9)^* \cdot (0 + \dots + 9)^*)$



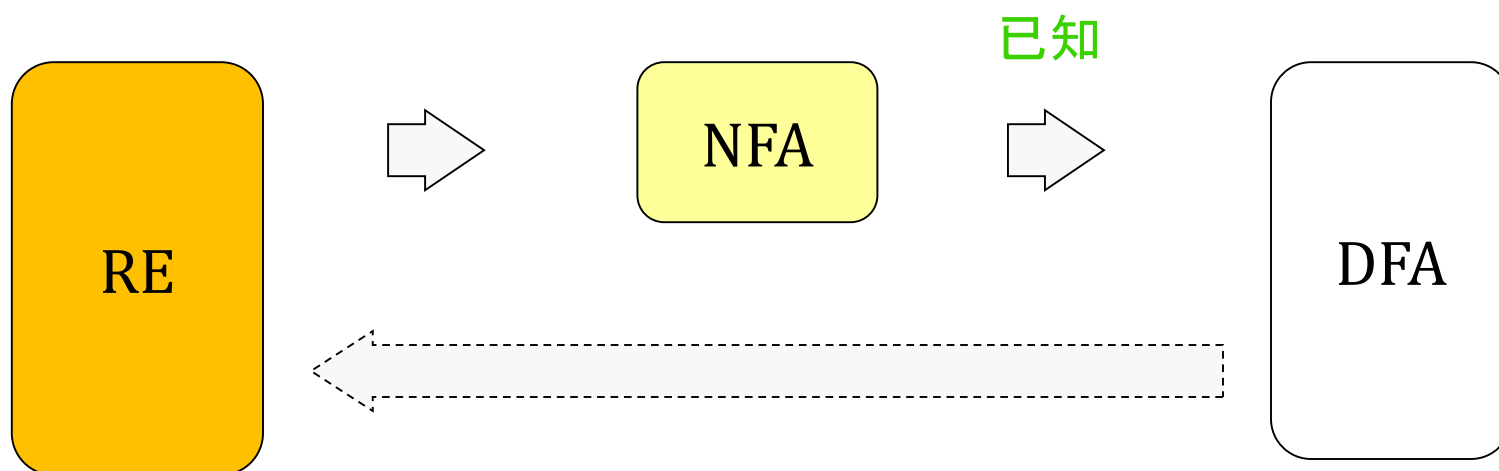
3.2 RE与DFA、NFA等价性

- RL:(识别器:(FA RE RG) 判定性质(δ 模式匹配 生成) 等价性质 (**RE $\equiv$ FA** RE $\equiv$ RG NFA $\equiv$ DFA))
- 定理3.1: 对任一DFA A 存在正则表达式 R 使得 $L(R)=L(A)$
- 定理3.2: 对任意正则表达式 R 存在 ε -NFA E 使得 $L(E)=L(R)$





定理3.2证明路线图

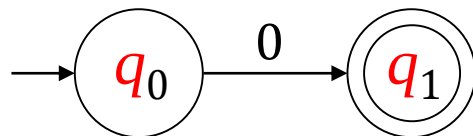


每一个用正则表达式来定义的语言也可用有穷自动机来定义



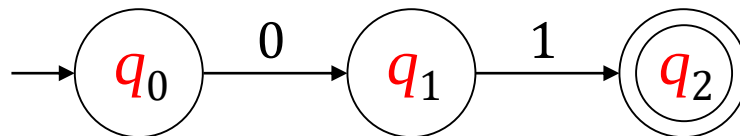
例：从正则表达式到 NFA

$$R_1 = 0$$



原子语言

$$R_2 = 01$$

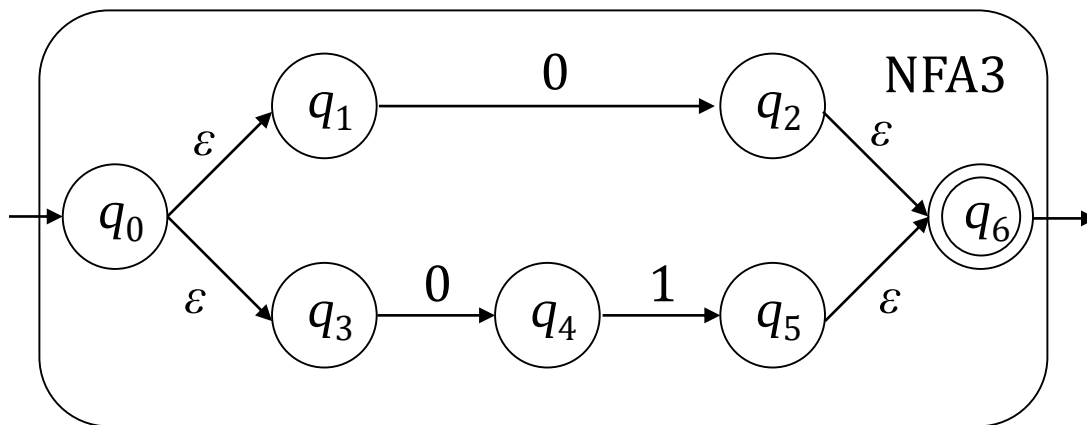


连接运算



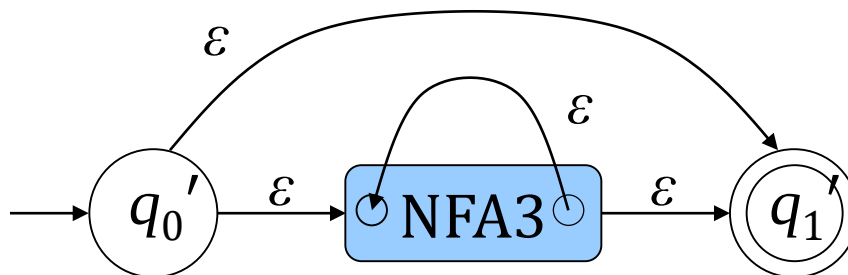
例：从正则表达式到 NFA

$$R_3 = 0 + 01$$



并

$$R_4 = (0 + 01)^*$$



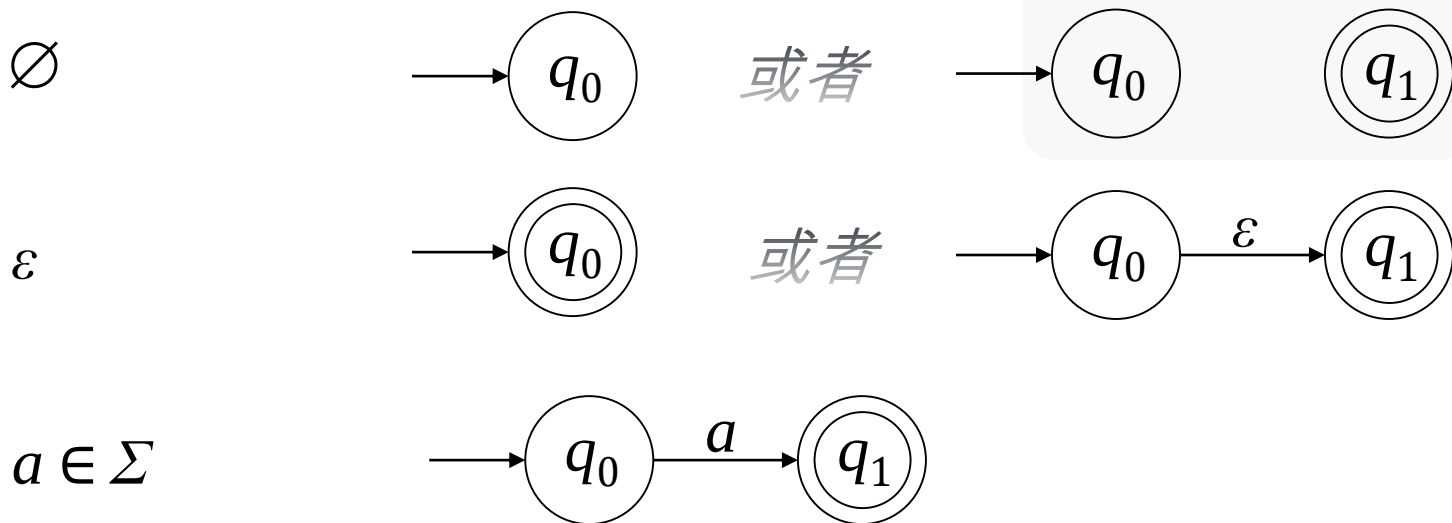
闭包



正则表达式转换为NFA通用方法

正则表达式

NFA



基础：

- 正则表达式 $\emptyset$, ε 和 $a, a \in \Sigma$ 对应的NFA如图所示。

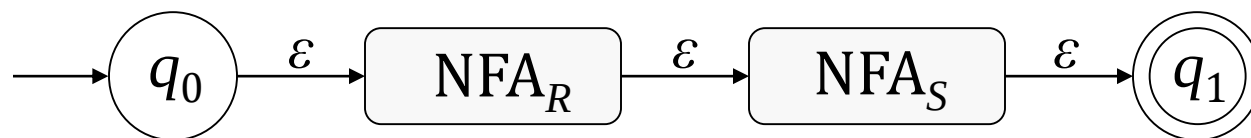
归纳

- 对于正则表达式 R 和 S , $R+S, RS, R^*$ 和 (R) , ?

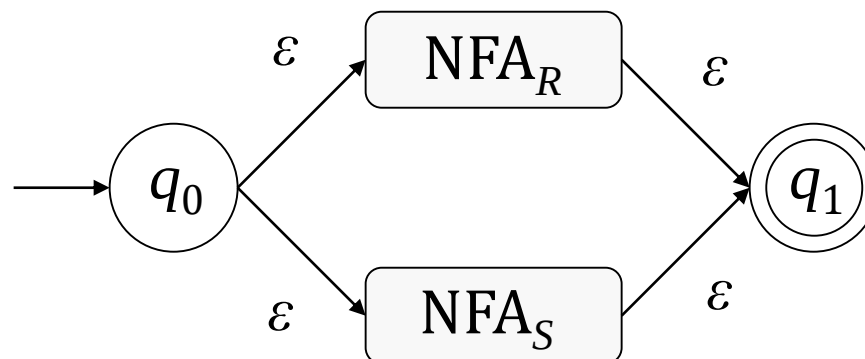


归纳证明示意

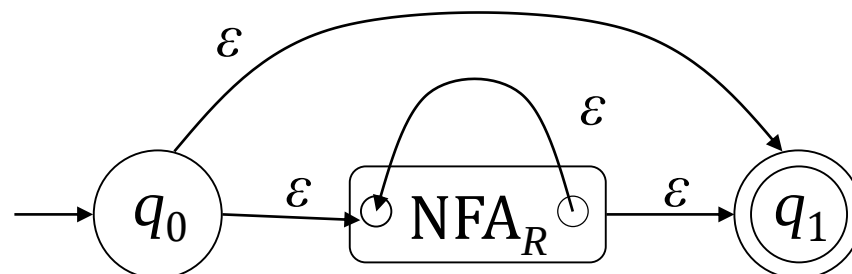
RS



$R+S$



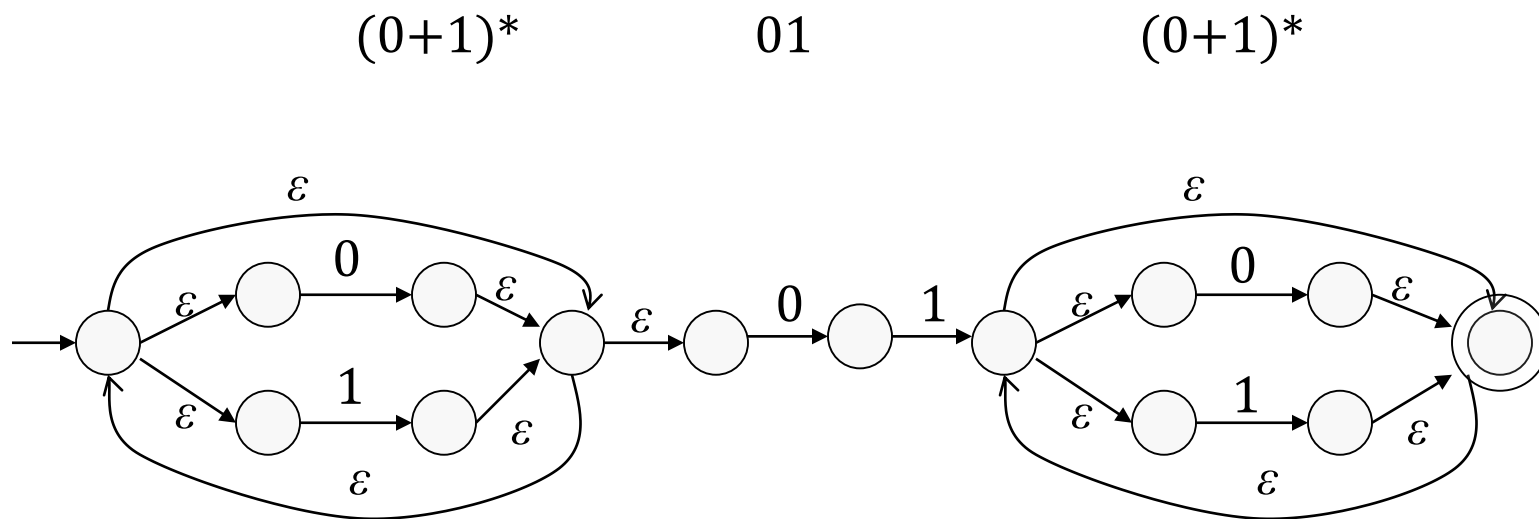
R^*





例：从RE构造NFA

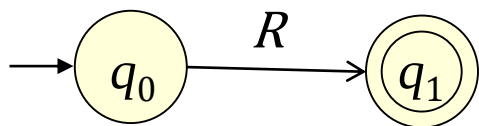
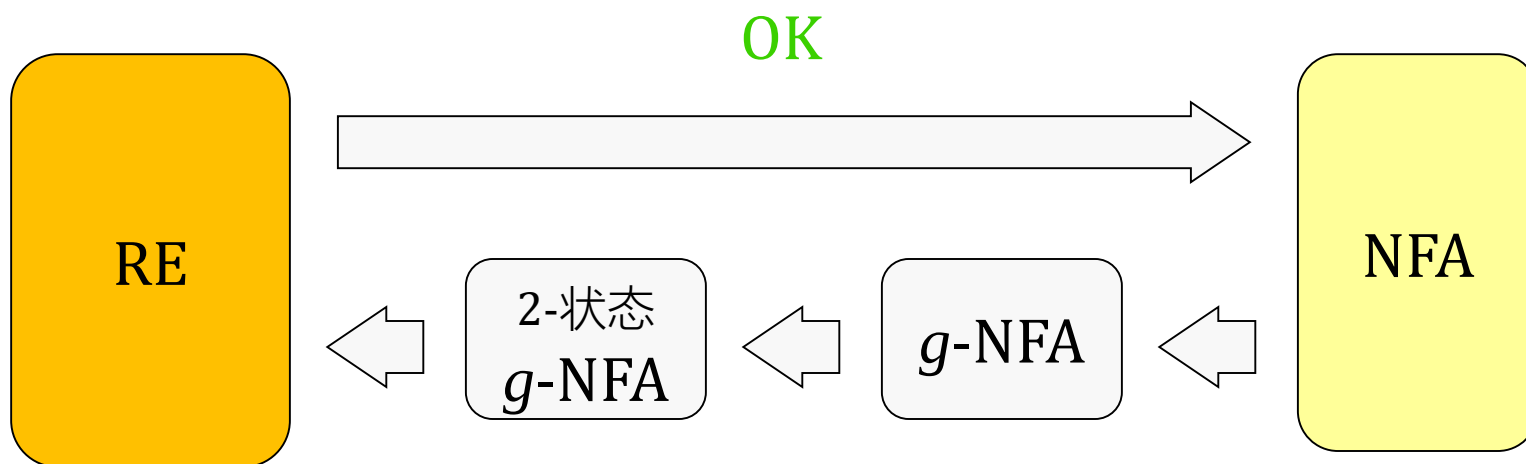
- $(0+1)^*01(0+1)^*$





有穷自动机转换为RE

- 对任一DFA A 存在正则表达式 R 使得 $L(R)=L(A)$
- DFA是NFA特例，所以关键是NFA到RE转换方法

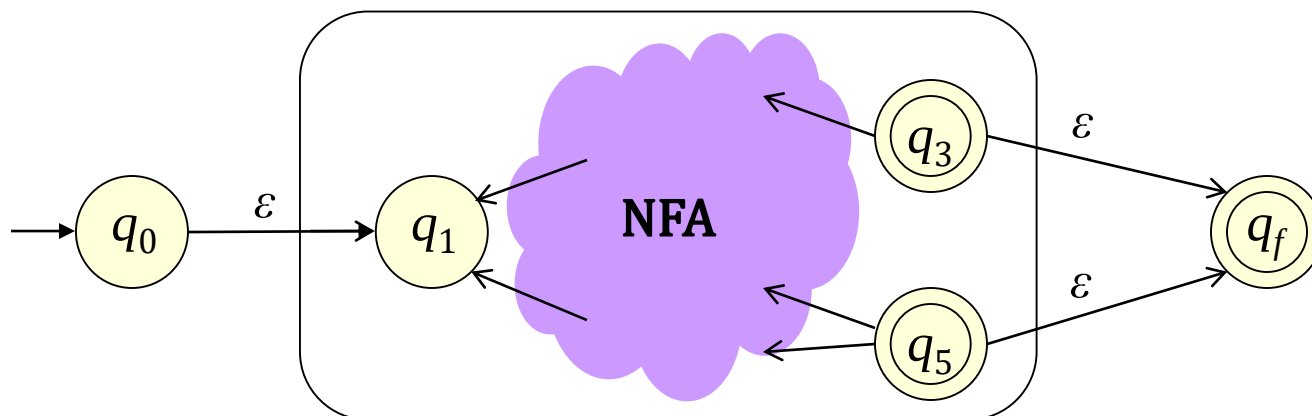


任一个 2-状态 g -NFA 总与某个正则表达式 R 等价。



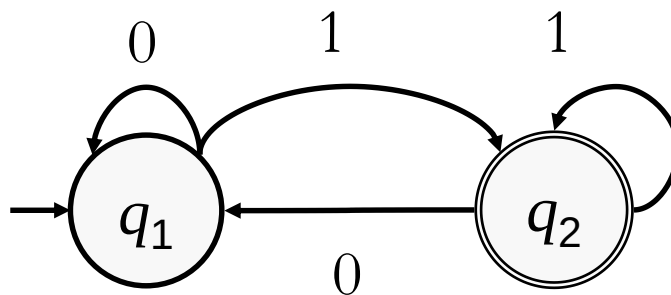
通用NFA (g -NFA)

- 只有一个接受状态
- 没有射入到初始状态的弧
- 没有从接受状态射出的弧
- 弧上标记采用正则表达式作标记





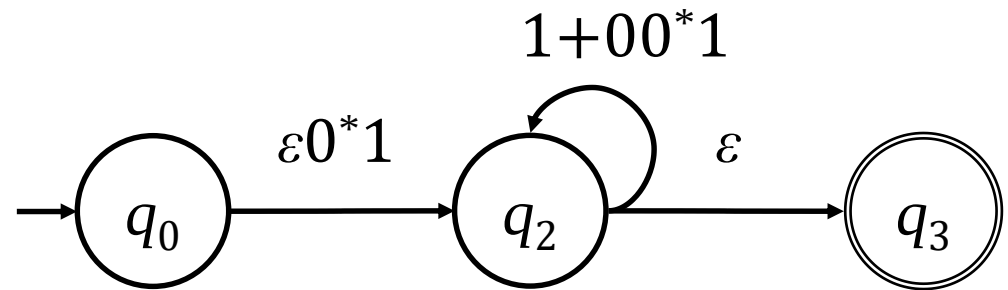
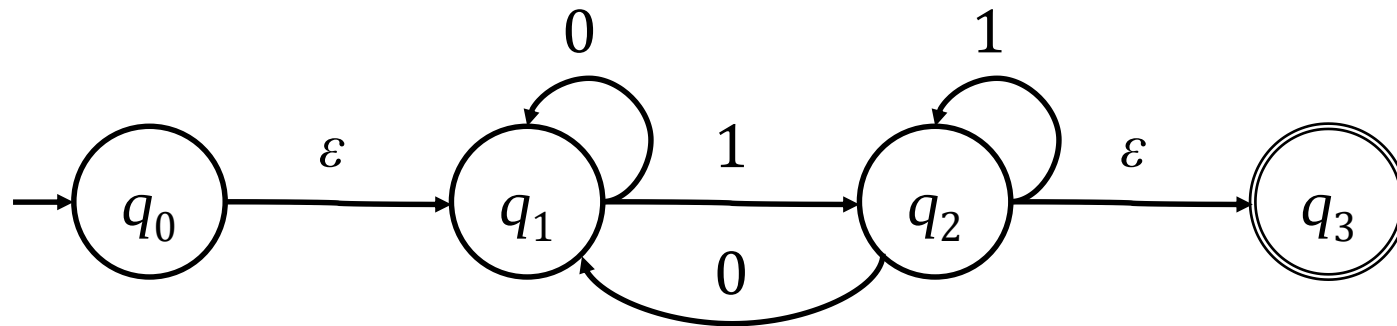
g -NFA



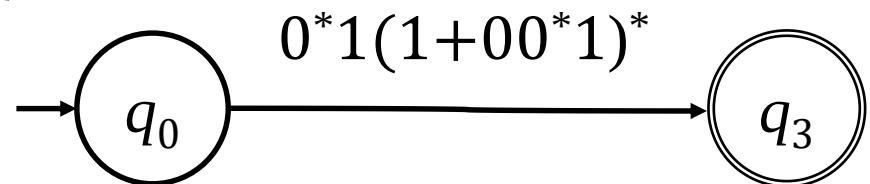
- 只有一个接受状态？ YES
- 没有射入到初始状态的弧？ NO
- 没有从接受状态射出的弧？ NO
- 弧上标记是正则表达式？ NO/YES



g -NFA



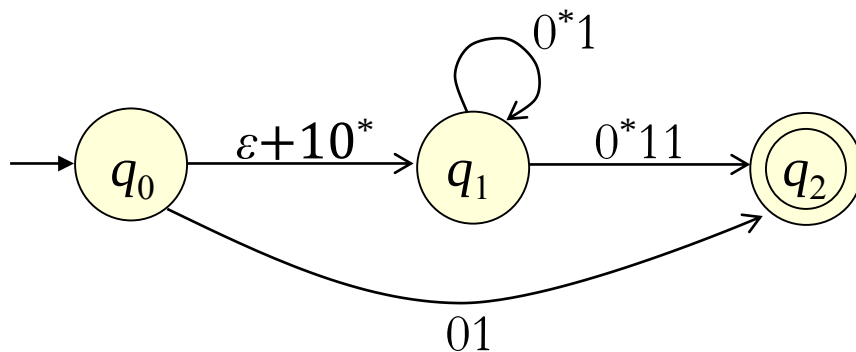
- ✓ - 只有一个接受状态
- ✓ - 没有射入到初始状态的弧
- ✓ - 没有从接受状态射出的弧





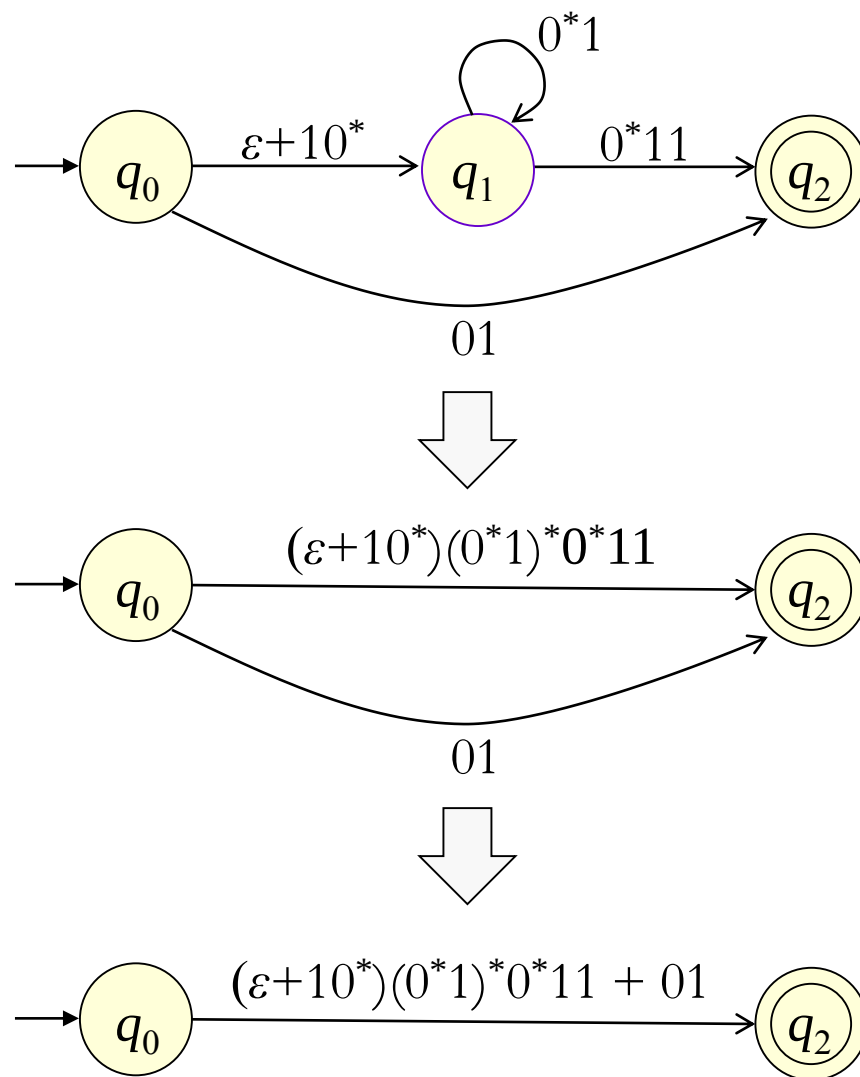
NFA与 g -NFA对比

- g -NFA弧上标记采用正则表达式作标记
- NFA的 a 弧看作正则表达式 a 做标记
- NFA的 ε 弧看作正则表达式 ε 做标记
- NFA的缺失弧可看作是正则表达式 $\emptyset$ 做标记的弧
- 如此，NFA弧上标记都被当作正则表达式，再检查没有射入初始状态的弧和没有从接受状态射出的弧，即成为 g -NFA





g -NFA的状态约简

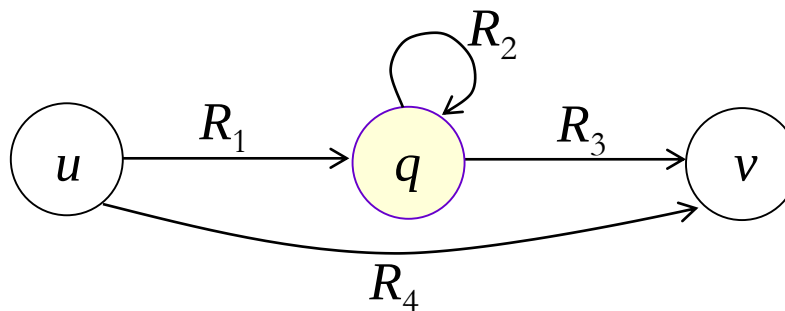




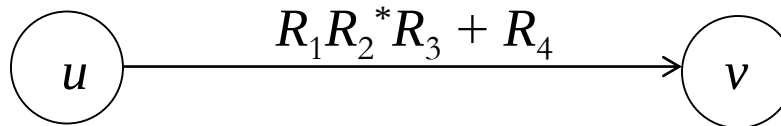
g -NFA状态约简——通用规则

- 要消除状态 q , 对于每个状态对 (u, v)

替代



为



- 注意 $u=v$ 时也如此

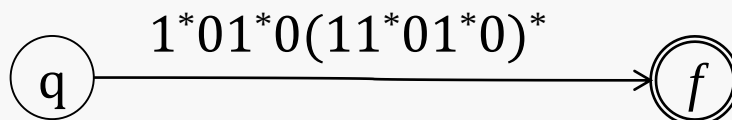
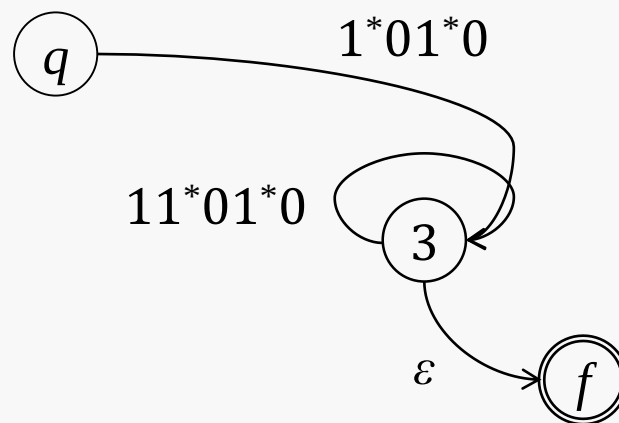
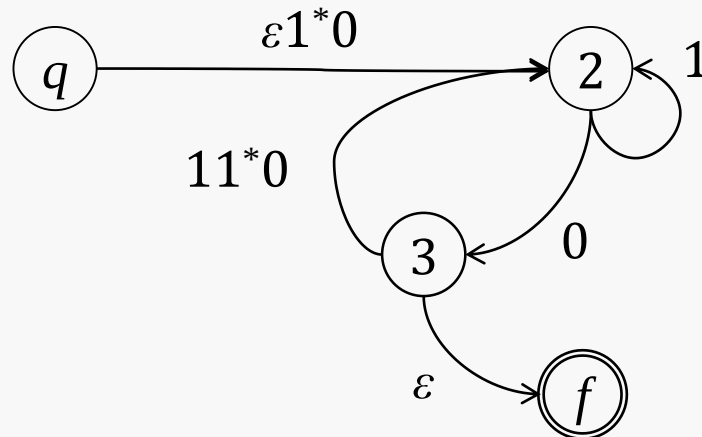
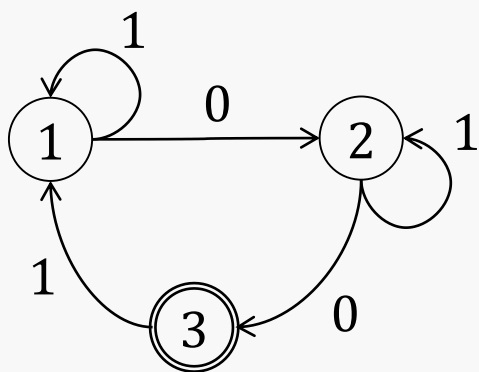
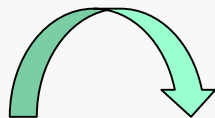
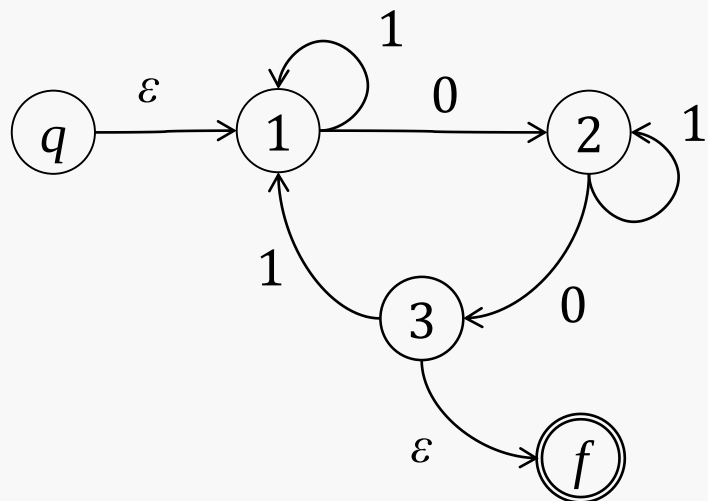
对于要消除的节点 s 进行不无遗漏的处理:

$\lambda(s)$ 头为 s 的弧尾的集合, 自环除外, 比如 u

$\mu(s)$ 尾为 s 的弧头的集合, 自环除外, 比如 v



例: g -NFA状态约简





定理3.1证明

- 定理3.1 对于任意一个DFA A , 存在一个正则表达式 R , 使得 $L(A) = L(R)$ 。
- DFA $A = (Q, \Sigma, v, q_0, F)$
- $L(A) = \{w \mid w \in \Sigma^*, \tilde{v}(q_0, w) \in F\}$
- 证明的思路:
 - 考察 q_0 至 q , $q \in F$, 之间所有路径, 用正则表达式表示之, 即得。
 - 这些路径是否不无遗漏地都表示为正则表达式了呢?



关键思路：用归纳法穷尽所有路径

- 对结点编号，对两端点间所有路径进行排序以便于归纳。
- 对DFA状态图中的顶点从1到 n ， $n = |Q|$ ，编号。
- 对始端 i 到末端 j 的所有路径， $1 \leq i, j \leq n$ ，进行如下排列：
 - 路径上除端点外的所有中间结点其编号不大于 k ，按照 $k = 0, 1, \dots, n$ 依次对路径进行排列
 - 注意 $k = 0$ 时满足条件的路径为只有端点，没有中间结点
- 那么针对路径 $\text{PATH}(i, j)$ 的归纳方式：
 - 基础：不经过任何中间结点的路径的标记为 $R_{ij}^{(0)}$
 - 归纳：经过不大于 k 的中间结点的路径的标记为
$$R_{ij}^{(k-1)} + R_{ik}^{(k-1)} (R_{kk}^{(k-1)})^* R_{kj}^{(k-1)} = R_{ij}^{(k)}$$



证明定理3.1

- 基础：不经过任何中间结点的路径为 $R_{ij}^{(0)}$ ，分两种情形：
- 情形一： $i \neq j$;
 - $R_{ij}^{(0)} = \emptyset$;
 - for $(a \in \Sigma)$ if $(j \in v(i, a))$ $R_{ij}^{(0)} = R_{ij}^{(0)} + a$;
- 情形二： $i = j$;
 - $R_{ij}^{(0)} = \varepsilon$;
 - for $(a \in \Sigma)$ if $(j \in v(i, a))$ $R_{ij}^{(0)} = R_{ij}^{(0)} + a$;
- 归纳：



证明定理3.1

- 归纳：假定顶点 i 到顶点 j 所有经过顶点不大于 $k-1$ 的路径，它们的标记并在一起定义为 $R_{ij}^{(k-1)}$ ，那么 i 到 j 所有经过顶点不大于 k 的路径，分为两种情形：
- 情形一：没有经过顶点 k ；
 - 根据归纳假设，这些路径的标记都属于 $L(R_{ij}^{(k-1)})$
- 情形二：经过顶点 k ；
 - 路径分段为：PATH(i, k), PATH(k, k), PATH(k, j), 且各段都属于情形一
 - 根据归纳假设，这三段组合成的所有路径，它们的标记构成的集合定义为 $R_{ik}^{(k-1)}(R_{kk}^{(k-1)})^*R_{kj}^{(k-1)}$
- 最后，将两种情形合并在一起得到
 - $R_{ij}^{(k)} = R_{ij}^{(k-1)} + R_{ik}^{(k-1)}(R_{kk}^{(k-1)})^*R_{kj}^{(k-1)}$



证明定理3.4

- 令 $n = |Q|$, Q 中元素编号为从1到 n , 其中 q_0 编号为1,
- 令 $r = +j \in \{q \text{ 的编号} \mid q \in F\} \cdot R_{1j}^{(n)}$
 - 即 $r = R_{1j_1}^{(n)} + \dots + R_{1j_m}^{(n)}$, 其中 $j_1, \dots, j_m$ 是 F 中各元素的编号
- 则 $L(A) = L(r)$
- 证毕。



例：构造DFA的正则表达式

- $R_{ij}^{(k)} = R_{ij}^{(k-1)} + R_{ik}^{(k-1)}(R_{kk}^{(k-1)})^* R_{kj}^{(k-1)}$
- $r = R_{13}^{(3)} = R_{13}^{(2)} + R_{13}^{(2)}(R_{33}^{(2)})^* R_{33}^{(2)}$

$$R_{13}^{(2)} = R_{13}^{(1)} + R_{12}^{(1)}(R_{22}^{(1)})^* R_{23}^{(1)}$$

$$R_{13}^{(1)} = \emptyset$$

$$R_{12}^{(1)} = R_{12}^{(0)} + R_{11}^{(0)}(R_{11}^{(0)})^* R_{12}^{(0)}$$

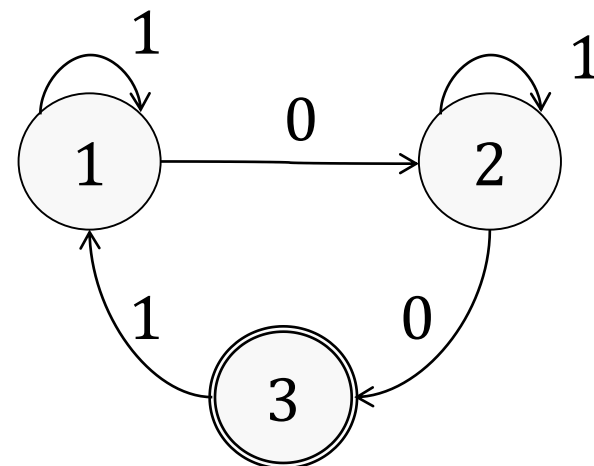
$$R_{22}^{(1)} = R_{22}^{(0)}$$

$$R_{23}^{(1)} = R_{23}^{(0)}$$

$$R_{33}^{(2)} = R_{33}^{(1)} + R_{32}^{(1)}(R_{22}^{(1)})^* R_{23}^{(1)}$$

$$R_{33}^{(1)} = R_{33}^{(0)} + R_{31}^{(0)}(R_{11}^{(0)})^* R_{13}^{(0)}$$

$$R_{32}^{(1)} = R_{32}^{(0)} + R_{31}^{(0)}(R_{11}^{(0)})^* R_{12}^{(0)}$$



$$R_{11}^{(0)} = \epsilon + 1$$

$$R_{12}^{(0)} = 0$$

$$R_{22}^{(0)} = \epsilon + 1$$

$$R_{23}^{(0)} = 0$$

$$R_{33}^{(0)} = \epsilon$$

$$R_{13}^{(0)} = \emptyset$$

$$R_{31}^{(0)} = 1$$

$$R_{32}^{(0)} = \emptyset$$

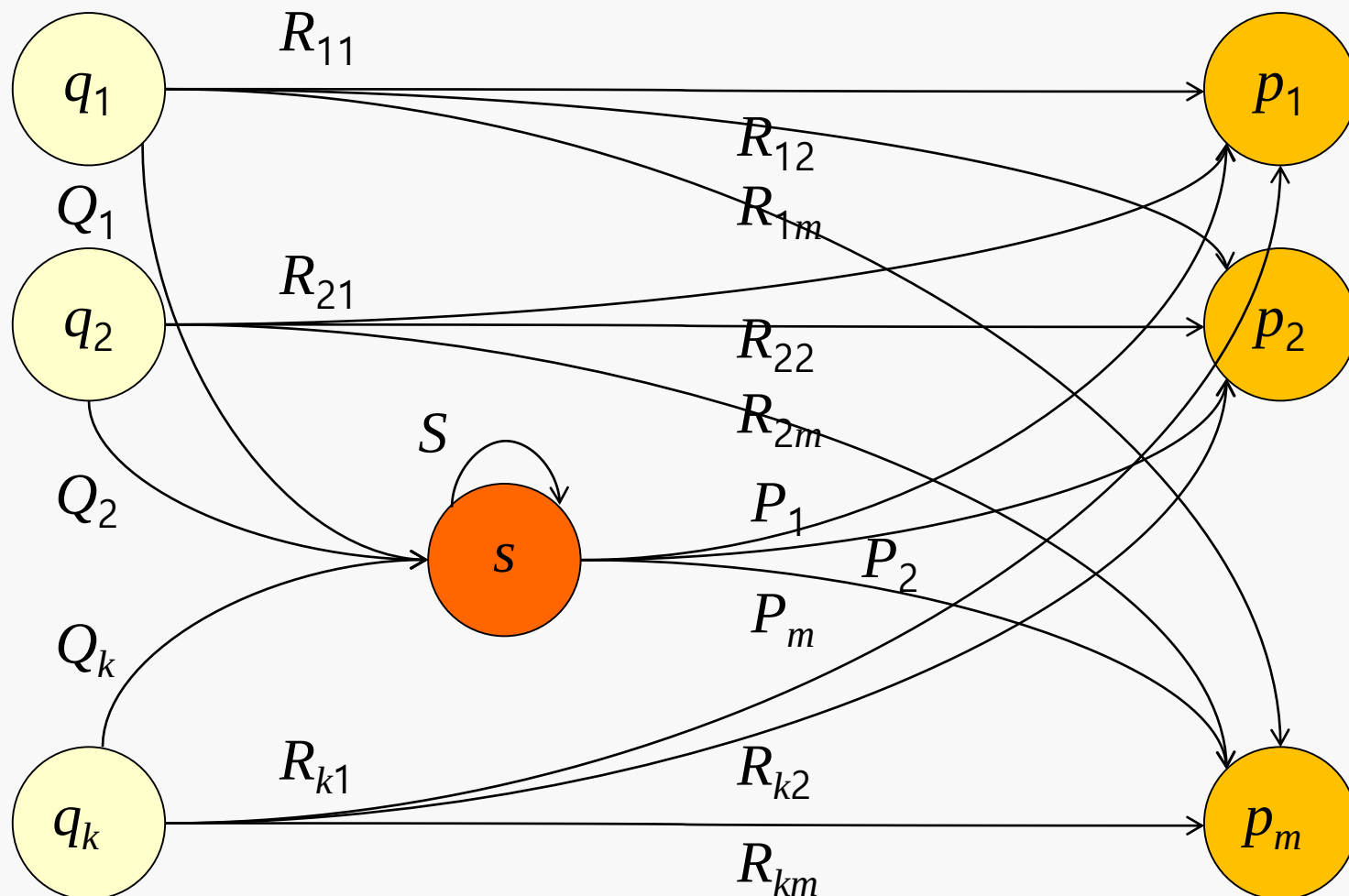
$$\emptyset + (0 + (\epsilon + 1)^* 0)(\epsilon + 1 + (\epsilon + 1)^*) (0) + (R_{33}^{(2)})^*$$

$$= (0 + 1^* 0)(\epsilon + 1^*) (0) + (R_{33}^{(2)})^*$$



状态约简规则 (消去s状态前)

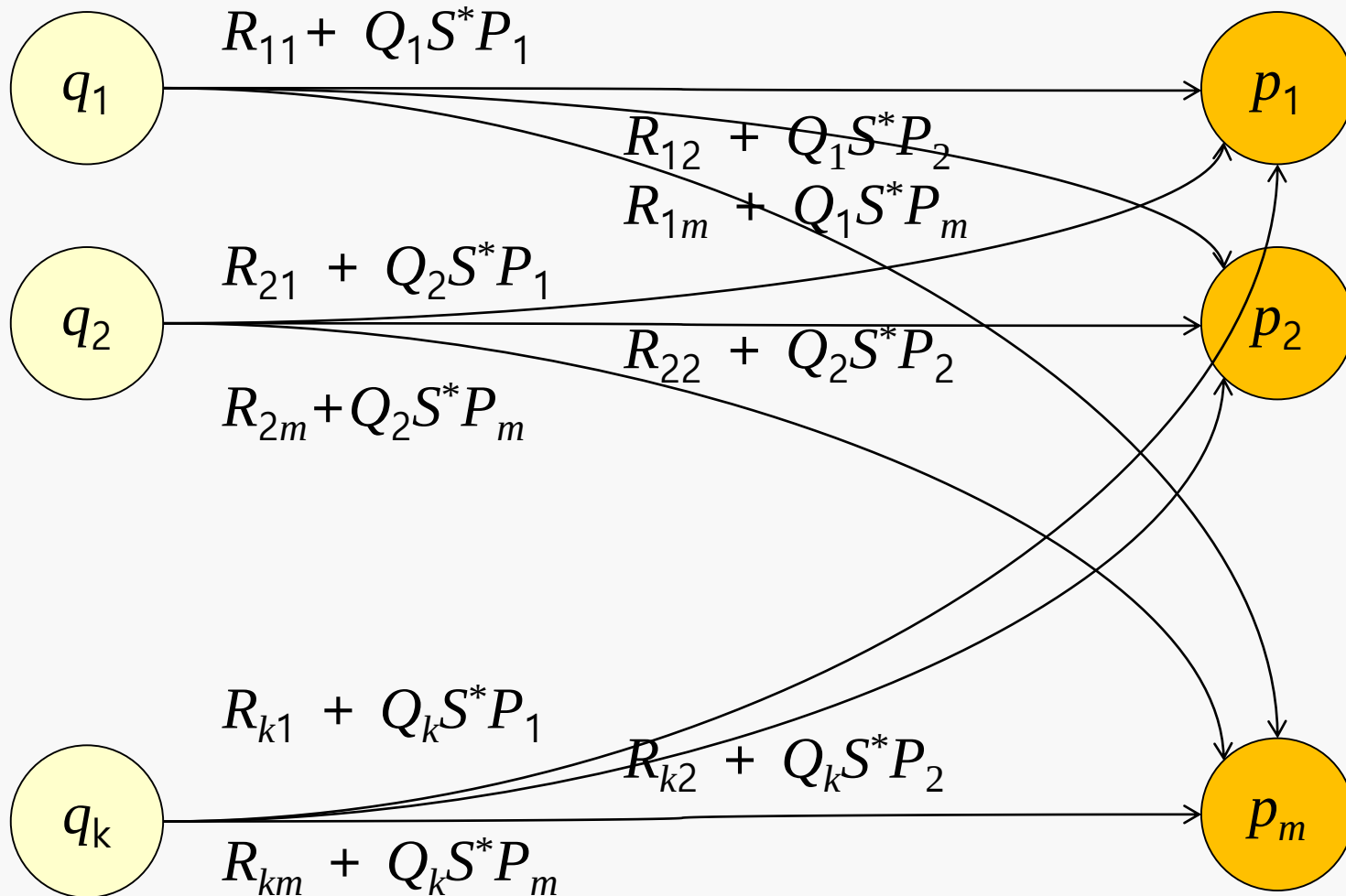
$$\lambda(s) = \{q_1, q_2, \dots, q_k\}; \mu(s) = \{p_1, p_2, \dots, p_m\}$$





状态约简规则 (消除s状态后)

$$\forall q \in \lambda(s), p \in \mu(s) \cdot R_{qp} + R_{qs} R_{ss}^* R_{sp}$$

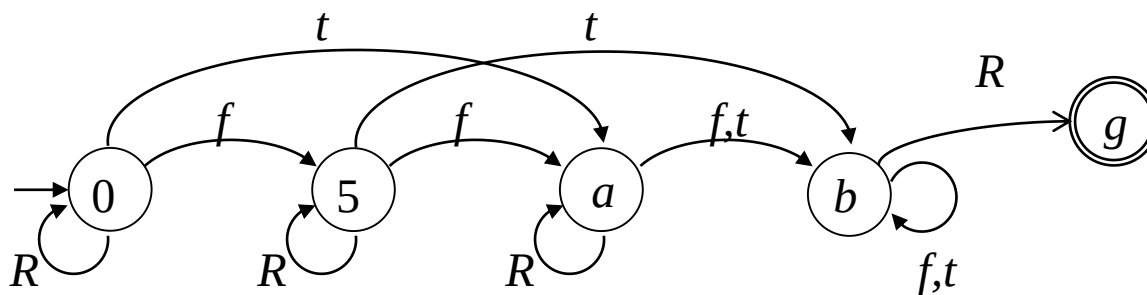




口香糖球机的语言

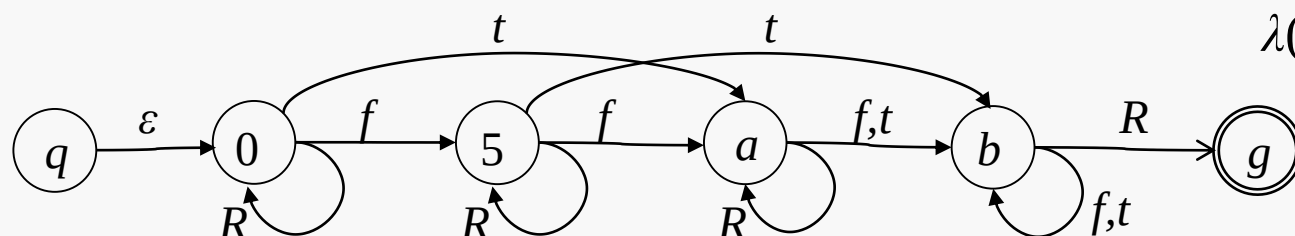
- 始端0末端 g 的路径有三类：
 - $0,5,a,b,g$ $R^*fR^*fR^*(f+t)(f+t)^*R$
 - $0,a,b,g$ $R^*tR^*(f+t)(f+t)^*R$
 - $0,5,b,g$ $R^*fR^*t(f+t)^*R$

$$R^*fR^*fR^*(f+t)(f+t)^*R + R^*tR^*(f+t)(f+t)^*R + R^*fR^*t(f+t)^*R$$





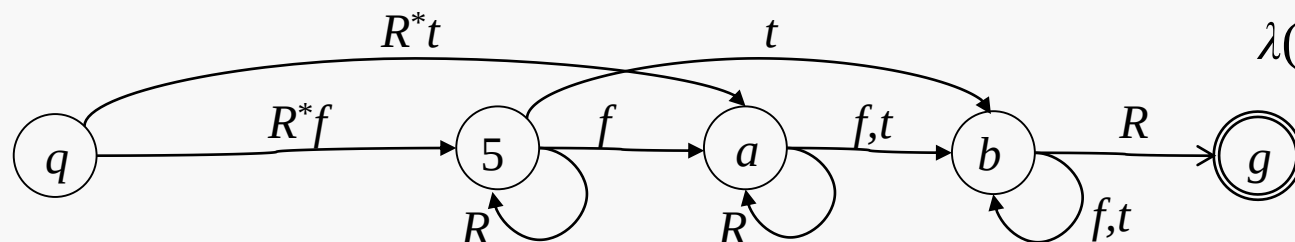
求口香糖球机的RE



$$\lambda(0) = \{q\}; \mu(0) = \{5, a\}$$

$$R_{q5} + R_{q0}R_{00}^*R_{05}$$

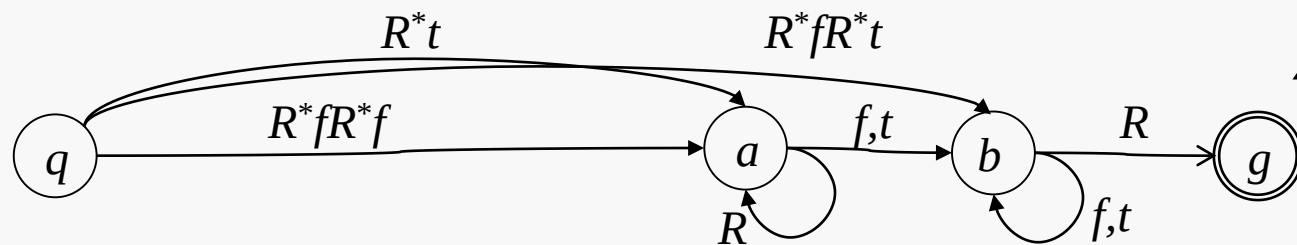
$$R_{qa} + R_{q0}R_{00}^*R_{0a}$$



$$\lambda(5) = \{q\}; \mu(5) = \{a, b\}$$

$$R_{qa} + R_{q5}R_{55}^*R_{5a}$$

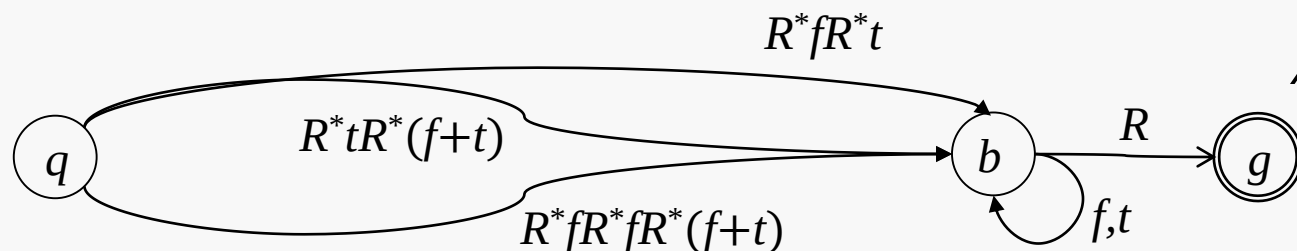
$$R_{qb} + R_{q5}R_{55}^*R_{5b}$$



$$\lambda(a) = \{q\}; \mu(a) = \{b\}$$

$$R_{qb} + R_{qa}R_{aa}^*R_{ab}$$

$$R_{qa} = ?$$



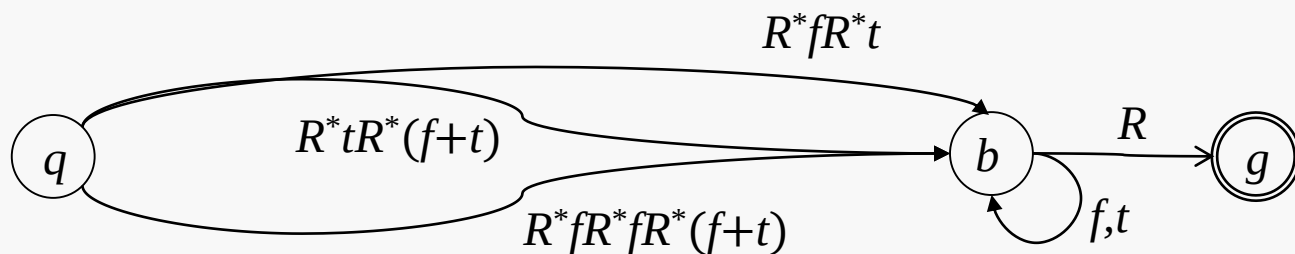
$$\lambda(b) = \{q\}; \mu(b) = \{g\}$$

$$R_{qg} + R_{qb}R_{bb}^*R_{bg}$$

$$R_{qb} = ?$$



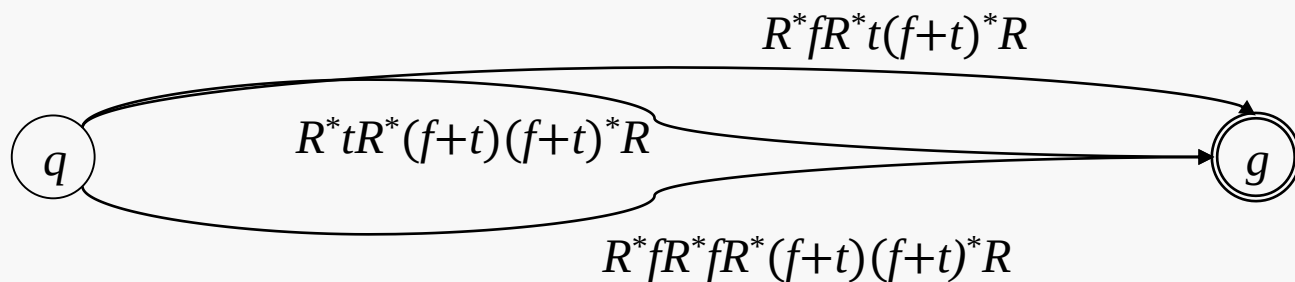
求口香糖球机的RE



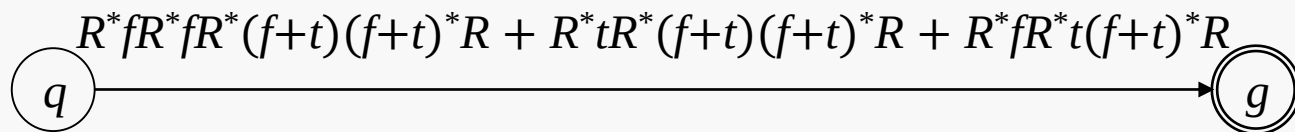
$$\lambda(b) = \{q\}; \mu(b) = \{g\}$$

$$R_{qg} + R_{qb}R_{bb}^*R_{bg}$$

$$R_{qb} = ?$$



$$R_{qg} = ?$$





3.3 正则表达式应用举例

- 正则表达式是基于模式匹配定义了实际应用中感兴趣的符号串集合，可用于生成、检索、识别有用的文本内容。通常地，这种作为模式的正则表达式被自动构建成**DFA**以方便程序实现来完成上述功能。
- 编译器工具Lex和Flex等，UNIX工具grep和awk等，都以正则表达式作为所匹配的文本的模式，生成词法分析器、或直接处理文本，得到广泛应用。
- 第4章介绍Lex



RE的常见应用范畴

- 编译器工具Lex和Flex等，UNIX工具grep和awk等，都以正则表达式作为所匹配的文本的模式，生成词法分析器、或直接处理文本，得到广泛应用。
- 第4章介绍Lex

部分常见的正则表达式元符号扩充

元符号	含义	示例
.	匹配任意单个字符，除换行符	a.c 匹配 abc/a1c/a c
[]	字符集合，匹配其中任意一个	[0-9] 数字, [a-zA-Z]字母
\	转义符，将字符转为普通字符	\. 小数点, * 星号
{n}/{n ,}/{n,m}	对前子表达式的数量限定	a{3,} 匹配 aaaa
+	匹配前子表达式1到多次	ab+ 匹配 ab..., 等价于ab{1,}
?	匹配前子表达式0次或1次	ab? 匹配 a/ab, 等价于ab{0,1}
\s	匹配空白符	\s+ 匹配多个空白



识别有用的文本内容

- 整数（无无效0）： $[-+]?[1-9][0-9]^* | 0$
- 整数： $[-+]?[0-9]^+$
- 定点数： $[-+]?[0-9]^+\.[0-9]^+$
- 数学定点数： $[-+]?([0-9]^+\.[0-9]^+|\.[0-9]^+|[0-9]^+\.|0)$
- 整数部分为0小数部分占8位的定点数： $[-+]?0\.[0-9]\{8\}$
- 浮点数（科学计数法）：
 - $[-+]?((([0-9]^+)|([0-9]^*\.[0-9]^+)|([0-9]^+\.[0-9]^*))([eE]([-+]?[0-9]^+)?$
- 浮点数（使用已知变量）： $<\text{数学定点数}> ([eE]<\text{整数}>)?$



识别有用的文本内容

- 实际中正则表达式常用于文本搜索，如识别街道地址等
- Street | St\.| Avenue | Ave\.| Road | Rd\.
- 对于有些街道包含多个单词，如：西安市的咸宁西路（Xianning West Rd.），华盛顿特区的罗德岛大道（Rhode Island Avenue）等，可采用：

`([A-Z][a-z]*\s) ([A-Z][a-z]*\s)* (Street | St\.| Avenue | Ave\.| Road | Rd\.)`

- 若地址上有门牌号码，
如28 Xianning West Road, 1200NW Rhode Island Ave.等，
那么只需在左边加上`[0-9]+[A-Z]{0,2}\s`即可。

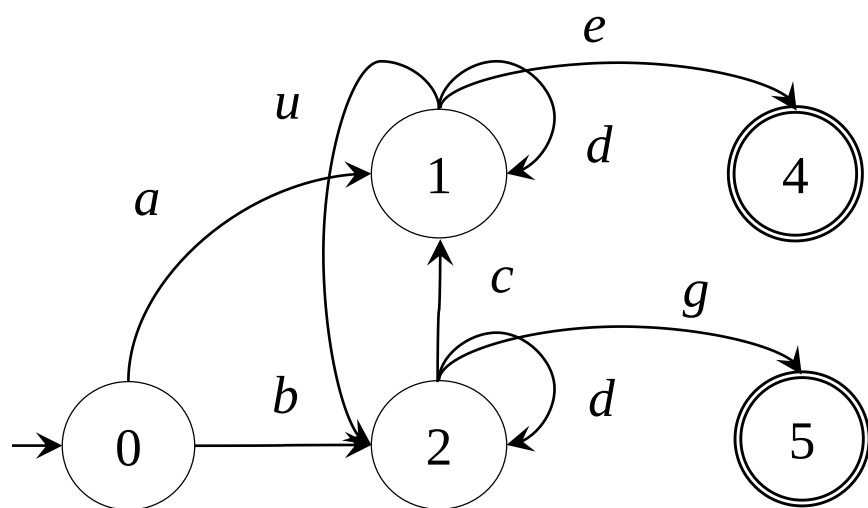


小结

- 知识点：RE定义，代数性质，元符号扩充，RL簇上的基本运算，DFA2转RE，RE转 ε -NFA，g-NFA
- 习题3.1(2)；习题3.2(2)(3)；习题3.4(2)

将DFA转为RE

2018



$$\lambda(1)=\{0, 2\}; \mu(1)=\{2, 4\}$$

$$R_{02}+R_{01}R_{11}^*R_{12}=b+ad^*u$$

$$R_{04}+R_{01}R_{11}^*R_{14}=\emptyset+ad^*e$$

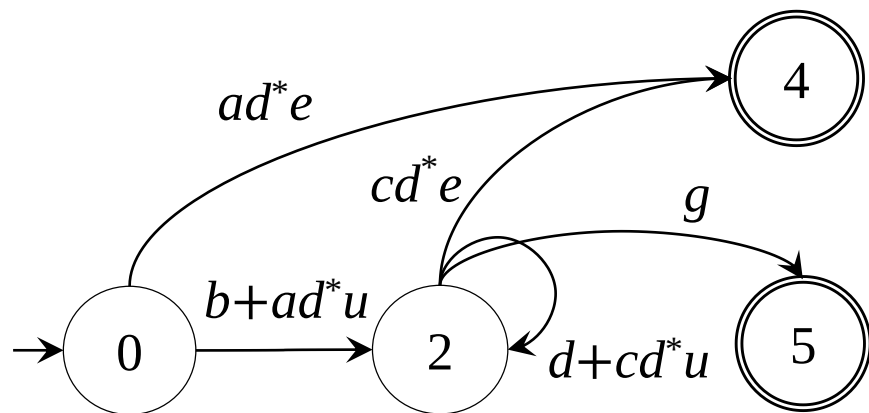
$$R_{22}+R_{21}R_{11}^*R_{12}=d+cd^*u$$

$$R_{24}+R_{21}R_{11}^*R_{14}=\emptyset+cd^*e$$

$$\lambda(2)=\{0\}; \mu(2)=\{4, 5\}$$

$$R_{04}+R_{02}R_{22}^*R_{24}=ad^*e+(b+ad^*u)(d+cd^*u)^*cd^*e$$

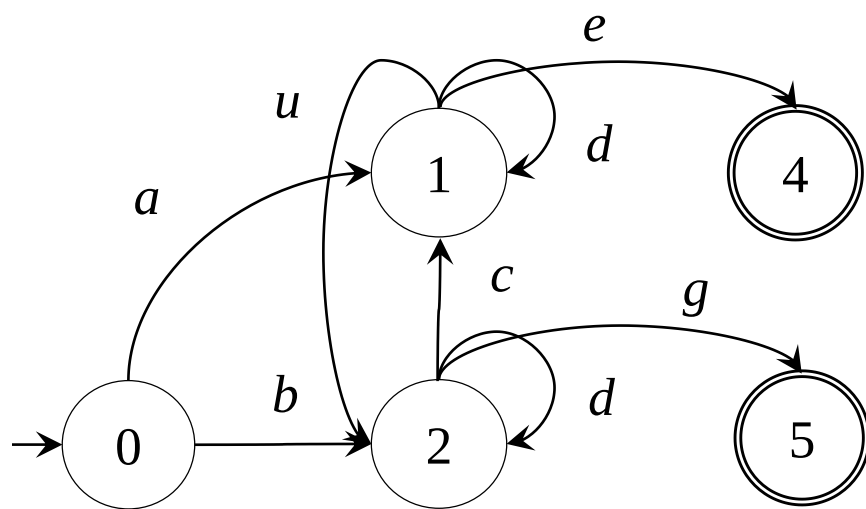
$$R_{05}+R_{02}R_{22}^*R_{25}=\emptyset+(b+ad^*u)(d+cd^*u)^*g$$



$$ad^*e+(b+ad^*u)(d+cd^*u)^*cd^*e+(b+ad^*u)(d+cd^*u)^*g$$

$$a(d+u(d+ud^*c)^*e)^*+b(d+cd^*u)^*g$$

将DFA转为RE



$$\lambda(2)=\{0, 1\}; \mu(2)=\{1, 5\}$$

$$R_{01}+R_{02}R_{22}^*R_{21}=a+bd^*c$$

$$R_{05}+R_{02}R_{22}^*R_{25}=\emptyset+bd^*g$$

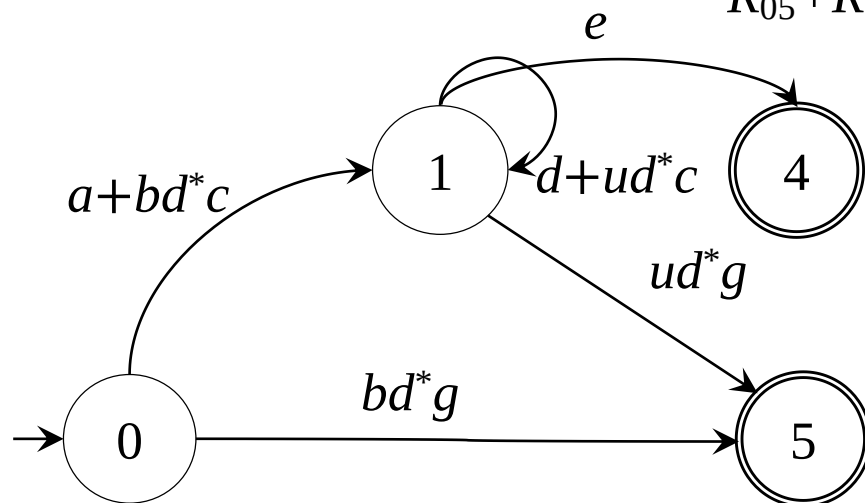
$$R_{11}+R_{12}R_{22}^*R_{21}=d+ud^*c$$

$$R_{15}+R_{12}R_{22}^*R_{25}=\emptyset+ud^*g$$

$$\lambda(1)=\{0\}; \mu(1)=\{4, 5\}$$

$$R_{04}+R_{01}R_{11}^*R_{14}=\emptyset+(a+bd^*c)(d+ud^*c)^*e$$

$$R_{05}+R_{01}R_{11}^*R_{15}=bd^*g+(a+bd^*c)(d+ud^*c)^*ud^*g$$



$$(a+bd^*c)(d+ud^*c)^*e+bd^*g+(a+bd^*c)(d+ud^*c)^*ud^*g$$

$$ad^*e+(b+ad^*u)(d+cd^*u)^*cd^*e+(b+ad^*u)(d+cd^*u)^*g$$



对比分析

$$\textcircled{1} (a+bd^*c)(d+ud^*c)^*e+bd^*g+(a+bd^*c)(d+ud^*c)^*ud^*g$$

$$\textcircled{2} ad^*e+(b+ad^*u)(d+cd^*u)^*cd^*e+(b+ad^*u)(d+cd^*u)^*g$$

$$\textcircled{1} ad^*e+b(d+cd^*u)^*cd^*e+ad^*u(d+cd^*u)^*cd^*e+b(d+cd^*u)^*g+ad^*u(d+cd^*u)^*g$$

$$\textcircled{2} a(d+ud^*c)^*e+bd^*c(d+ud^*c)^*e+bd^*g+a(d+ud^*c)^*ud^*g+bd^*c(d+ud^*c)^*ud^*g$$

$$\textcircled{1} b(d^*cd^*ud^*)^*cd^*e$$

$$\textcircled{2} bd^*c(d^*ud^*cd^*)^*e$$